

Laboratorio di Calcolo

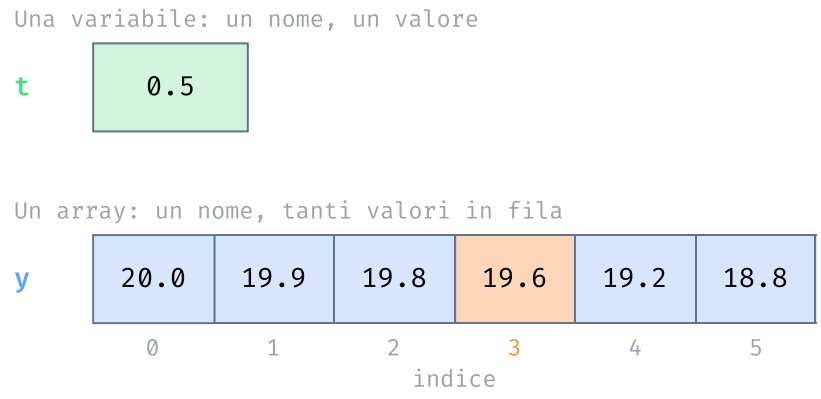
A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Array monodimensionali

Che cos'è un array?

- *Array* è una parola inglese: vuol dire **fila ordinata**, **schieramento**. In italiano si dice anche **vettore**.
- Finora una variabile era una **scatola** con un nome, che contiene **un solo** valore.
- Un array è una **fila di scatole** tutte dello **stesso tipo**, una accanto all'altra, con un **unico nome**.
- Ogni scatola si indica con il suo **numero di posizione**, l'**indice**: si conta da **0**.
- È come una colonna del quaderno di laboratorio: "la misura numero 3".



`y[3]` è la scatola numero 3 dell'array `y`, cioè la **quarta**: vale 19.6.

Perché servono gli array?

In laboratorio non si misura una grandezza **una volta sola**: si fanno decine o migliaia di misure.

```c
double y0, y1, y2, y3, y4, y5;  // posizioni [m]

scanf("%lf", &y0);
scanf("%lf", &y1);
scanf("%lf", &y2);
// ... e se le misure fossero 1000?
```

- Con una variabile per misura il codice cresce con il numero di dati.
- Un ciclo **non può** scorrere `y0`, `y1`, `y2` …: sono nomi diversi, non un "indice".

```c
double y[10];                    // 10 posizioni [m]
int i;

for (i = 0; i < 10; i++) {
    scanf("%lf", &y[i]);
}
```

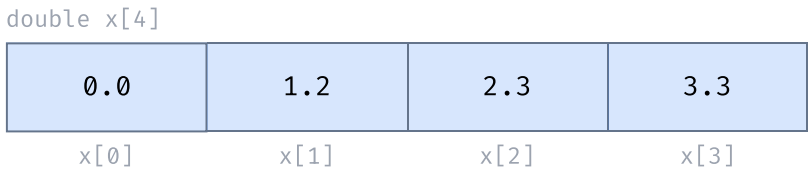
- Un **array** (o **vettore**) è una sequenza di variabili dello **stesso tipo**, con un **unico nome**.
- Ogni elemento si individua con un **indice** intero: `y[i]`.
- L'indice è un'espressione: lo possiamo far variare in un **ciclo** (lezione 15).

Dichiarazione di un array

```
tipo nome[dimensione];
```

```
double x[4];        // 4 double
int    conteggi[10]; // 10 int
```

- La **dimensione** è il numero di elementi; è fissata nella dichiarazione e **non cambia** più.
- Tutti gli elementi hanno lo **stesso tipo**.
- Gli elementi sono numerati da **0** a **dimensione − 1**: `x[0]`, `x[1]`, `x[2]`, `x[3]`.



Le quadre hanno due significati: nella **dichiarazione** `double x[4]` il numero è la **dimensione**; nell'**uso** `x[3]` è l'**indice**. `x[4]` non esiste!

Eppure il programma compila e lo legge lo stesso: `printf("%g", x[4])` stampa `2.14592e-314` su una macchina, `-nan` su un'altra. È il contenuto della memoria **dopo** l'array.

L’array in memoria

Gli elementi di un array occupano celle di memoria **consecutive** (memoria **contigua**).

- Come per le variabili (lezione 10), ogni elemento ha un **indirizzo**.
- Un `double` occupa 8 byte: gli indirizzi degli elementi crescono di **8** in 8.
- L’elemento `x[i]` si trova a:

$$\text{indirizzo di } x[0] + i \times \texttt{sizeof(double)}$$

- Per questo l’accesso a `x[i]` è **immediato**, qualunque sia `i`.
- L’indice è uno **spostamento** dall’inizio: ecco perché si parte da **0**.

Memoria (schematica): `double x[4]`

0×7ffc08	`int n = 4`
0×7ffc10	`x[0] = 0.0`
0×7ffc18	`x[1] = 1.2`
0×7ffc20	`x[2] = 2.3`
0×7ffc28	`x[3] = 3.3`
0×7ffc30	`double t = 0.5`

Prima e dopo l’array ci possono essere **altre variabili**: il C non mette "recinti" attorno all’array.

Accesso agli elementi

```c
#include <stdio.h>

int main() {
    double x[4];          // 4 posizioni [m]

    x[0] = 0.0;
    x[1] = 1.2;
    x[2] = 2.3;
    x[3] = x[2] + 1.0;    // un elemento si usa come una variabile

    int i = 2;
    printf("primo:   %.1f\n", x[0]);
    printf("ultimo:  %.1f\n", x[3]);
    printf("x[i]:    %.1f\n", x[i]);
    printf("x[i+1]:  %.1f\n", x[i + 1]);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o indici indici.c
$ ./indici
primo:   0.0
ultimo:  3.3
x[i]:    2.3
x[i+1]:  3.3
```

- `x[i]` si usa **esattamente** come una variabile `double`: in un'espressione, a sinistra di `=`, con `+=`, `++` …
- L'indice può essere una **qualunque espressione intera**: `x[i + 1]`, `x[2 * k]`, `x[n - 1]`.
- L'indice deve essere **intero**: `x[1.5]` è un errore di compilazione.

Inizializzazione

```c
int    a[5] = {3, 1, 4, 1, 5};
int    b[5] = {7, 8};          // parziale
double c[]  = {0.5, 1.5, 2.5}; // dimensione dedotta: 3
int    z[4] = {0};             // tutti a zero
int    j[5] = {11};            // tutti a 11?

printf("%d %d %d %d %d\n", a[0], a[1], a[2], a[3], a[4]);
printf("%d %d %d %d %d\n", b[0], b[1], b[2], b[3], b[4]);
printf("%.1f %.1f %.1f\n", c[0], c[1], c[2]);
printf("%d %d %d %d\n", z[0], z[1], z[2], z[3]);
printf("%d %d %d %d %d\n", j[0], j[1], j[2], j[3], j[4]);
```

```
3 1 4 1 5
7 8 0 0 0
0.5 1.5 2.5
0 0 0 0
11 0 0 0 0
```

Un array **non inizializzato** (`double x[4];`) contiene valori **indefiniti**, come una variabile non inizializzata (lezione 10).

- I valori tra graffe vanno agli elementi **nell'ordine**.
- Se i valori sono **meno** della dimensione, gli elementi restanti valgono **0**.
- `= {0}` è il modo standard per azzerare tutto l'array.
- Ma `= {11}` **non** mette tutto a 11: solo `j[0]` vale 11, gli altri 0.
- Se si omette la dimensione, il compilatore la **deduce** contando i valori.
- Le graffe si possono usare **solo nella dichiarazione**: dopo, `a = {1, 2, 3};` è un errore.

Attraversare un array con for

Il ciclo `for` con un contatore da `0` a `N - 1` è il modo naturale di visitare **tutti** gli elementi.

```c
#include <stdio.h>

#define N 6

int main() {
    const double g  = 9.81;   // [m/s^2]
    const double dt = 0.1;    // [s]
    double t[N], y[N];
    int i;

    for (i = 0; i < N; i++) {
        t[i] = i * dt;
        y[i] = 0.5 * g * t[i] * t[i];
    }

    for (i = 0; i < N; i++) {
        printf("%d  %.2f  %.4f\n", i, t[i], y[i]);
    }
    return 0;
}
```

```
$ ./moto
0  0.00  0.0000
1  0.10  0.0491
2  0.20  0.1962
3  0.30  0.4415
4  0.40  0.7848
5  0.50  1.2263
```

- Caduta libera da fermo, campionata ogni $\Delta t = 0.1$ s: $y_i = \frac{1}{2} g t_i^2$.
- La condizione è `i < N`, **non** `i <= N`: l'ultimo indice è `N - 1`.
- `t[i]` e `y[i]` con lo **stesso indice** descrivono lo stesso istante.
- **Test:** $y_5 = \frac{1}{2} \cdot 9.81 \cdot 0.25 = 1.22625$ ✓

La dimensione: una costante simbolica

```c
#define N 100

double t[N], y[N];

for (i = 0; i < N; i++) {
    ...
}
```

- La dimensione compare in **molti punti**: dichiarazioni, cicli, calcoli.
- Con `#define` (lezione 13) la si cambia in **un solo punto**.
- Si può anche fissare in compilazione: `gcc -DN=1000 ...` (togliendo il `#define`, oppure proteggendolo con `#ifndef N`).

Da evitare

```c
double t[100], y[100];

for (i = 0; i < 100; i++) { ... }
for (i = 0; i < 99; i++)  { ... }
```

Se si passa da 100 a 200 misure, bisogna trovare **tutti** i `100` (e i `99`!).

Anche `const int n = 100; double y[n];` compila, ma crea un array "a lunghezza variabile" che **non** si può inizializzare con `= {…}`. Per ora usate `#define`.

Quanti elementi ha un array?

L'operatore `sizeof` (lezione 10) applicato a un array dà la dimensione **in byte** dell'intero array.

```c
#include <stdio.h>

int main() {
    double m[] = {9.8, 9.7, 9.9, 9.8, 9.6};
    int n = sizeof(m) / sizeof(m[0]);

    printf("sizeof(m)    = %zu\n", sizeof(m));
    printf("sizeof(m[0]) = %zu\n", sizeof(m[0]));
    printf("elementi     = %d\n", n);
    return 0;
}
```

```
$ ./dimensione
sizeof(m)    = 40
sizeof(m[0]) = 8
elementi     = 5
```

- Byte totali diviso byte di un elemento = **numero di elementi**: $40/8 = 5$.
- Utile con la dimensione **dedotta** (`m[]`): aggiungendo un valore tra le graffe, `n` si aggiorna da solo.
- Funziona solo **dove l'array è dichiarato**: quando passeremo gli array alle funzioni questo trucco non varrà più.

Fuori dai limiti

```c
#include <stdio.h>

#define N 5

int main() {
    int conteggi[N] = {0};
    int altro = 100;
    int i;

    for (i = 0; i <= N; i++) {  // <= invece di <
        conteggi[i] = i;
    }
    printf("altro = %d\n", altro);
    printf("conteggi[7] = %d\n", conteggi[7]);
    return 0;
}
```

Comportamento indefinito: `altro` può restare 100 oppure essere sovrascritto, e `conteggi[7]` vale un numero diverso a ogni esecuzione; il programma può anche terminare con *Segmentation fault*.

- Il C **non controlla** gli indici: `conteggi[5]` e `conteggi[7]` leggono e scrivono la memoria **dopo** l'array.
- Il programma compila e spesso "funziona": l'errore resta nascosto e i risultati sono sbagliati.
- Il compilatore a volte se ne accorge, con indici **costanti** e soprattutto ottimizzando (`-O2`):

```
fuori.c:14:34: warning: array subscript 7
  is above array bounds of ‘int[5]’
  [-Warray-bounds=]
```

- Nel ciclo con `i <= N`, di solito **tace**. Controllate sempre gli estremi dei cicli!

Copiare un array

```c
double a[N] = {1.0, 2.0, 3.0};
double b[N];

b = a;
```

```
$ gcc -Wall -Wextra -o copia copia.c
copia.c:9:7: error: assignment to expression
  with array type
```

- Tra array **non** esiste l'assegnamento: `b = a` non copia gli elementi.
- Allo stesso modo `a == b` **non** confronta i contenuti.
- Il nome dell'array indica il blocco di memoria, non i valori.

```c
double a[N] = {1.0, 2.0, 3.0};
double b[N];
int i;

for (i = 0; i < N; i++) {
    b[i] = a[i];
}
b[0] = 99.0;              // cambia solo b
printf("a: %.1f %.1f %.1f\n", a[0], a[1], a[2]);
printf("b: %.1f %.1f %.1f\n", b[0], b[1], b[2]);
```

```
a: 1.0 2.0 3.0
b: 99.0 2.0 3.0
```

- Si copia **elemento per elemento**, con un ciclo.
- Dopo la copia i due array sono **indipendenti**.

Leggere un vettore di misure

```c
#include <stdio.h>

#define N 6

int main() {
    double y[N];   // posizioni [m]
    int i;

    for (i = 0; i < N; i++) {
        scanf("%lf", &y[i]);
    }
    for (i = N - 1; i >= 0; i--) {
        printf("y[%d] = %.3f\n", i, y[i]);
    }
    return 0;
}
```

- `&y[i]`: come per una variabile, `scanf` vuole l'indirizzo dell'elemento (lezione 13).
- Il secondo ciclo scorre l'array **all'indietro**.

```
$ cat caduta.txt
0.000
0.049
0.196
0.442
0.785
1.226
$ ./leggi < caduta.txt
y[5] = 1.226
y[4] = 0.785
y[3] = 0.442
y[2] = 0.196
y[1] = 0.049
y[0] = 0.000
```

- Con `<` il programma legge da un **file** invece che dalla tastiera (è il simmetrico di `>`, lezione 12): `scanf` non vede la differenza.

Dalle posizioni all'accelerazione

```c
#include <stdio.h>
#define N 6

int main() {
    const double dt = 0.1;   // [s]
    double y[N];             // posizioni [m]
    double v[N - 1];         // velocità [m/s]
    double a[N - 2];         // accelerazioni [m/s^2]
    int i;

    for (i = 0; i < N; i++) {
        scanf("%lf", &y[i]);
    }
    for (i = 0; i < N - 1; i++) {
        v[i] = (y[i + 1] - y[i]) / dt;
    }
    for (i = 0; i < N - 2; i++) {
        a[i] = (v[i + 1] - v[i]) / dt;
        printf("v = %.2f m/s  a = %.1f m/s^2\n",
               v[i], a[i]);
    }
    return 0;
}
```

```
$ ./velocita < caduta.txt
v = 0.49 m/s  a = 9.8 m/s^2
v = 1.47 m/s  a = 9.9 m/s^2
v = 2.46 m/s  a = 9.7 m/s^2
v = 3.43 m/s  a = 9.8 m/s^2
```

- Posizioni di caduta libera arrotondate al **mm** (come lette da un video).
- $v_i = (y_{i+1} - y_i)/\Delta t$: da N posizioni, $N-1$ velocità e $N-2$ accelerazioni.
- Con `i < N` nel secondo ciclo, `y[i + 1]` uscirebbe dall'array!
- a oscilla attorno a g per gli arrotondamenti: come farne la **media**? Prossima lezione.

Errori tipici con gli array

Errore	Cosa succede
Ciclo con `i <= N`	Si accede a `x[N]`, che non esiste: comportamento indefinito
Indice che parte da 1	Si salta `x[0]` e si esce dall'array all'ultimo passo
`y[i + 1]` con `i` fino a `N - 1`	Fuori dai limiti all'ultimo passo: il ciclo deve fermarsi a `N - 2`
Array non inizializzato usato come accumulatore	Valori di partenza casuali: usate `= {0}`
`b = a;` per copiare	Errore di compilazione: copiate con un ciclo
`a = {1, 2, 3};` dopo la dichiarazione	Errore: le graffe valgono solo nell'inizializzazione
`scanf("%lf", y[i])` senza `&`	`-Wall` dà un warning `-Wformat`; in esecuzione, crash o dati persi

Gli errori sugli indici **non** vengono segnalati dal compilatore: il **test** su un caso noto (lezione 7) è l'unico modo per scoprirli.

Cosa stampa?

Quanto vale ciascuna espressione? Pensateci **prima** di andare avanti.

```c
int    a[5] = {2, 4, 6};
int    b[]  = {1, 2, 3, 4};
double c[3];
int    i = 1;

int s = 0;
for (i = 0; i < 5; i++) {
    s += a[i];
}
```

Espressione	Valore	Perché
`a[3]`	`0`	Inizializzazione parziale
`a[1 + 1]`	`6`	È il terzo elemento
`sizeof(b)/sizeof(b[0])`	`4`	Dimensione dedotta
`s`	`12`	$2 + 4 + 6 + 0 + 0$